Παράλληλη Επεξεργασία Διανυσματικών Συζεύξεων στο Apache Spark: Μια Προσεγγιστική Προσέγγιση με Locality-Sensitive Hashing

ΠΕΡΙΛΗΨΗ

Η παρούσα εργασία πραγματεύεται μια εναλλακτική, πιο αποδοτική λύση στην εύρεση όμοιων διανυσμάτων ανάμεσα σε δύο σύνολα. Ένας εξαντλητικός έλεγχος όλων των ζευγαριών έχει τετραγωνική πολυπλοκότητα ενώ λύσεις όπως οι κλασικές δενδρικές δομές καταρρέουν σε πολλές διαστάσεις λόγω της "Κατάρας της Διαστατικότητας" (Curse of Dimensionality). Η προσέγγιση που ακολουθείται είναι προσεγγιστική μέσω Locality-Sensitive Hashing και κατανεμημένης επεξεργασίας στο Apache Spark. Η λύση αυτή συγκρίνει μόνο πιθανά ζευγάρια αποδίδοντας ~ 99% ανάκληση και τουλάχιστον 5,2x αποδοτικότητα σε 100 εκατομμύρια συγκρίσεις έναντι της κλασικής εξαντλητικής σύγκρισης (baseline). Διάφορες εφαρμογές μεγάλων δεδομένων επωφελούνται από αυτή την αποδοτικότητα όπως η ανάκτηση πολυμεσικού περιεχομένου, η ανάλυση κειμένου και τα συστήματα συστάσεων.

1. Εισαγωγή

Πολλές σύγχρονες εφαρμογές μεγάλων δεδομένων απαιτούν την εύρεση "ομοιών" διανυσμάτων που ανήκουν σε δύο διαφορετικά σύνολα, δηλαδή διανυσμάτων όπου η Ευκλείδεια απόσταση δεν υπερβαίνει μια δυναμική τιμή που ορίζει ο χρήστης. Τέτοιες εφαρμογές είναι η ανάκτηση πολυμεσικού περιεχομένου, η ανάλυση κειμένου και τα συστήματα συστάσεων [3].

Η μαθηματικός ορθή προσέγγιση του προβλήματος απαιτεί τον υπολογισμό του καρτεσιανού γινομένου και στην συνέχεια το φιλτράρισμα των αποστάσεων. Αυτή η προσέγγιση χαρακτηρίζετε από τετραγωνική πολυπλοκότητα $O(|R| \cdot |S|)$, όπου R και S δύο σύνολα d-διάστατων διανυσμάτων. Στην σύγχρονη εποχή τα σύνολα αυτά αποτελούνται από εκατομμύρια διανύσματα εκατοντάδων διαστάσεων, συνεπώς το πλήθος των συγκρίσεων που απαιτείτε είναι απαγορευτικό. Επιπλέον οι παραδοσιακές προσεγγίσεις με δενδρικές δομές ευρετηρίασης αδυνατούν να λύσουν αποδοτικά το πρόβλημα λόγο των πολλών διαστάσεων των διανυσμάτων, φαινόμενο γνωστό και ως «Κατάρα της Διαστατικότητας» (Curse of Dimensionality) [1].

Η παρούσα εργασία προτείνει μια λύση αυτού του προβλήματος. Η λύση βασίζεται στην ανάπτυξη ενός παράλληλου αλγορίθμου με Apache Spark με την τεχνική Locality-Sensitive Hashing (LSH) [2]. Το LSH δεν υπολογίζει όλες τις αποστάσεις, αντ΄αυτου μόνο τις αποστάσεις διανυσμάτων που έχουν υψηλή πιθανότητα να είναι "κοντά". Οι βασικές πυλώνες της υλοποίησης είναι οι εξής:

- Σχεδιασμός και υλοποίηση p-stable LSH για τον υπολογισμό της Ευκλείδειας απόστασης.
- Χρήση Apache Spark μέσω Map-Side Join [4] και Broadcast μεταβλητών στοχεύοντας στην εξάλειψη της συμφόρησης του δικτύου.
- Επεξεργασίας δεδομένων ανά τμήματα (chunks) για μείωση της απαιτούμενης μνήμης και χρήση βιβλιωθηκών χαμηλού επιπέδου (NumPy) για αύξηση ταχύτητας.
- Διεξαγωγή πειραμάτων στο σύνολο δεδομένων SIFT1M για βελτιστοποίηση της λύσης αλλά και απόδειξη της κλιμακοσιμότητας διατηρώντας σταθερά ανάκληση άνω του 98.5% .

Σημείωση: Η κατανεμημένη υλοποίηση έγινε στο Google Colab καθως δεν υπήρχε η δυνατότητα χρήσης του συστήματος ΩΚΕΑΝΟΣ.

2. Σχετικές Εργασίες

Η αναζήτηση όμοιων διανυσμάτων ήταν στο επίκεντρο της έρευνας για πολύ καιρό. Με την πάροδο του χρόνου οι χώροι των διανυσμάτων ξεκίνησαν να μεγαλώνουν οπότε η έρευνα οδηγήθηκε σε προσεγγιστικές μεθόδους με κυριότερη την τεχνική Locality-Sensitive Hashing (LSH) που εισήχθη από τους Indyk και Motwani [1]. Η αρχική προσέγγιση αφορούσε χώρους Hamming αλλά στην συνέχεια οι Datar κ.ά. [2] ανέπτυξαν τον p-stable LSH για την περίπτωση των χώρων με Ευκλείδεια απόσταση. Η συγκεκριμένη προσέγγιση αξιοποιεί τυχαίες προβολές δεδομένων μέσα από την κανονική κατανομή, αποτελώντας τη μαθηματική βάση της παρούσας υλοποίησης.

Όσον αφορά τα κατανεμημένα συστήματα οι κλασικές προσεγγίσεις του Map Reduce[3] καθιστούνται μη επαρκείς καθώς απαιτούν τη μεταφορά τεράστιων όγκων δεδομένων μέσω δικτύου. Συνεπώς η παρούσα εργασία εφαρμόζει τη λογική του Map-Side Join [4] στο σύγχρονο περιβάλλον In-Memory επεξεργασίας του Apache Spark [5] παρακάμπτοντας έτσι τη φάση ανακατανομής βελτιώνοντας την απόδοση και την κλιμακωσιμότητα της λύσης. Η χρήση του Map-Side Join είναι αποτελεσματική όταν το δεύτερο σύνολο είναι αρκετά μικρό για να χωρέσει στη μνήμη του κάθε κόμβου (broadcast).

3. ΠΕΡΙΓΡΑΦΗ ΑΛΓΟΡΙΘΜΟΥ ΚΑΙ ΥΛΟΠΟΙΗΣΗ

3.1 p-stable LSH για Ευκλείδεια Απόσταση

Η λογική του LSH βασίζεται στις συναρτήσεις κατακερματισμού (hash functions). Υπάρχουν διάφορες συναρτήσεις κατακερματισμού που αυξάνουν την πιθανότητα δύο διανύσματα που βρίσκονται κοντά στον γεωμετρικό χώρο να "συγκρουστούν", δηλαδή να έχουν το ίδιο αποτέλεσμα (hash)

όταν εφαρμοστεί η συνάρτηση κατακερματισμού σε αυτά. Στην παρούσα εργασία η απόσταση αξιολογείται με την Ευκλείδεια νόρμα (L2-norm) και σύμφωνα με τους Datar κ.ά. στη θεμελιώδη μελέτη τους [2], η συνάρτηση κατακερματισμού $h(v) = \left\lfloor \frac{a \cdot v + b}{w} \right\rfloor$

είναι η κατάλληλη. Το "a" είναι ένα τυχαίο διάνυσμα προβολής του οποίου οι τιμές προέρχονται από την τυπική κανονική κατανομή N(0,1), το "b" είναι μια τυχαία μετατόπιση που επιλέγεται από την ομοιόμορφη κατανομή στο [0, w], και το "w" αντιπροσωπεύει το πλάτος του κάδου (bucket width).

Η έννοια του κάδου (bucket) αντιστοιχεί σε ένα αριθμητικό διάστημα: η συνάρτηση προβάλλει (πολλαπλασιάζει) τα διανύσματα πάνω σε μια τυχαία ευθεία, στη συνέχεια η ευθεία "κόβεται" από την floor σε διακριτά, ίσα τμήματα μήκους w. Τα διανύσματα που καταλήγουν να "πέφτουν" στο ίδιο τμήμα, θεωρούνται ότι ανήκουν στον ίδιο κάδο.

Παρ' όλ' αυτά, η μεμονωμένη συνάρτηση δημιουργεί αρκετά ψευδώς θετικά αποτελέσματα. Για την αντιμετώπιση αυτού, συνδυάζονται K ανεξάρτητες συναρτήσεις κατακερματισμού, απαιτώντας ταυτόχρονη σύμπτωση. Επιπλέον, για τη μεγιστοποίηση της ανάκλησης, η διαδικασία επαναλαμβάνεται L φορές δημιουργώντας ανεξάρτητους πίνακες κατακερματισμού. Δύο διανύσματα είναι πιθανόν κοντά και ελέγχονται ως προς την ακριβή τους απόσταση στην περίπτωση που συμπέσουν στον ίδιο κάδο σε τουλάχιστον έναν από τους L πίνακες.

3.2 Αρχιτεκτονική Map-Side Broadcast Join

Στα πλαίσια του Apache Spark, ένα κλασικό Shuffle Join έχει απαγορευτικό κόστος δικτύου για την παρούσα εργασία. Ανακατανέμει όλα τα RDDs στους κόμβους εργασίας βασιζόμενο στα κλειδιά κατακερματισμού.

Συγκρίνοντας τα δύο σύνολα που καλούμαστε να διαχειριστούμε παρατηρούμε ότι το ένα είναι πολύ μικρότερο από το άλλο , μόλις 10.000 έναντι 1.000.000 διανυσμάτων. Συνεπώς χρησιμοποιήθηκε η λογική του Map-Side Broadcast Join. Το μικρό σύνολο S των 10.000 ερωτημάτων πηγαίνει αρχικά στον κεντρικό κόμβο (Driver), στον οποίο κατασκευάζεται το ευρετήριο κάδων (bucket index) για όλους τους L πίνακες. Το ευρετήριο αυτό στέλνεται στους workers οι οποίοι λειτουργούν αυτόνομα ελέγχοντας τα διανύσματα που έχουν απέναντι στο ευρετήριο. Με αυτόν τον τρόπο εκμηδενίζεται η διακίνηση δεδομένων στο δίκτυο κατά την εκτέλεση.

3.3 Επεξεργασία κατά Τμήματα (Chunking) και Vectorization

Με σκοπό την αποφυγή σφαλμάτων Out-Of-Memory, η επεξεργασία των διανυσμάτων του συνόλου R γίνεται τμηματικά (chunking) με μέγεθος 1.000 διανυσμάτων. Αυτός ο αριθμός μπορεί να προσαρμοστεί ανάλογα τη διαθέσιμη μνήμη που έχουν τα συστήματα που δουλεύουμε. Μέσω της NumPy κάθε τμήμα γίνεται ένας δισδιάστατος πίνακας βοηθώντας έτσι τον υπολογισμό των προβολών μέσω πολλαπλασιασμού πινάκων (vectorization). Αυτή η τεχνική επιταχύνει τη δημιουργία των hashes καθώς αξιοποιεί άμεσα τις διανυσματικές εντολές του επεξεργαστή.

4. ΠΕΙΡΑΜΑΤΙΚΗ ΑΠΟΤΙΜΗΣΗ

4.1 Σύνολο Δεδομένων & Περιβάλλον

Όπως υποδεικνύει η εργασία χρησιμοποιήθηκε το σύνολο δεδομένων SIFT1M από ένα σύνολο βάσης R με 1.000.000 διανύσματα και ένα σύνολο ερωτημάτων S με 10.000 διανύσματα. Όλα τα διανύσματα είναι διαστάσεων d=128 με τιμές τύπου float32. Το περιβάλλον εκτέλεσης των πειραμάτων ήταν το Google Colab το οποίο διαθέτει 2 υπολογιστικούς πυρήνες και 12 GB RAM. Η παράλληλη επεξεργασία στους 2 αυτούς πυρήνες υλοποιήθηκε μέσω του Apache Spark έκδοσης 4.0.2 σε τοπική λειτουργία. Ωστόσο οι πόροι που δώθηκαν στο Apache Spark περιορίστικαν στα 10 GB RAM, 6 για τον driver και 2 για τον κάθε worker. Αυτός ο περιορισμός έγινε με σκοπό τον έλεγχο της αποδοτικότητας της λύσης σε περιβάλλοντα με περιορισμένους πόρους, αλλά και για να μην υπάρχει πρόβλημα με το Google Colab.

Στις πειραματικές διαδικασίες που ακολούθησαν, το βέλτιστο πλήθος των συναρτήσεων κατακερματισμού (K) διατηρήθηκε. Από αρχικές δοκιμές διαπιστώθηκε ότι το K=4 προσφέρει την βέλτιστη ισορροπία μεταξύ ενός αυστηρού ορίου φιλτραρίσματος και της ανάκλησης.

4.2 Επίδραση Αριθμού Πινάκων (L)

Ο αριθμός των πινάκων κατακερματισμού L είναι άμεσα συνδεδεμένος με την ανάκληση που έχει ο αλγόριθμός μας οπότε κρίθηκε σκόπιμη η εξέταση της βέλτιστης τιμής. Διατηρώντας σταθερά το θ=200 (απόσταση ζευγαριών), το K=4, το w=300 (1,5 φορές το θ, πλάτος κάδου) εξετάστηκε η επίδραση του L στο δείγμα R=100.000, S=1.000. Η τιμή θ=200 επιλέχθηκε ως μια αντιπροσωπευτική μέση απόσταση για τον χώρο των 128 διαστάσεων του συγκεκριμένου συνόλου δεδομένων.

Πίνακας 1:Επίδραση του αριθμού πινάκων κατακερματισμού (L) στον χρόνο εκτέλεσης και την ανάκληση (K=4, w=300, θ=200).

L	Χρόνος LSH (s)	Recall (%)	Σύγκριση με Baseline
10	12,95	58,3%	17,9x
20	20,35	83,3%	13,3x
30	28,14	93,2%	9,3x
40	37,39	97,0%	7,1x
50	44,40	98,9%	5,8x
60	52,00	99,4%	5,0x

Από τα αποτελέσματα επιβεβαιώνεται ότι μικρό L συνεπάγεται χαμηλό recall. Καθώς το L αυξάνεται, το recall συγκλίνει και αυξάνεται ο χρόνος. Ιδανικό σημείο ισορροπίας επιλέγεται το L=50 καθώς υπερκαλύπτει τον στόχο του 95% χωρίς να προσθέτει περιττό υπολογιστικό φόρτο. Θα μπορούσε να επιλεγεί και το L=40 αλλά προτιμήθηκε το μεγαλύτερο recall έναντι της ταχύτητας.

4.3 Επίδραση Πλάτους Κάδου (w)

Η αυστηρότητα του φιλτραρίσματος ρυθμίζεται από το πλάτος του κάδου w. Κρίθηκε απαραίτητο λόγω της φύσης αυτής το w να μην είναι σταθερό αλλά να μεταβάλλεται σε σχέση με το θ πολλαπλασιαστικά (w_factor). Οι μετρήσεις διεξήχθησαν στο δείγμα 100.000 x 1.000, με L=50, K=4, θ=200, μεταβάλλοντας τον πολλαπλασιαστή του κατωφλίου.

Πίνακας 2: Επίδραση του πολλαπλασιαστή πλάτους κάδου (w factor) στην ανάκληση και τον χρόνο.

w factor	w	Recall (%)	Speedup vs Baseline
0,5x	100	13,7%	26,7x
1,0x	200	76,1%	13,4x
1,5x	300	98,9%	5,3x
2,0x	400	100%	2,6x
3,0x	600	100%	1,2x

Παρατηρούμε ότι για πολύ στενούς κάδους (w=0,5 x θ) ο χρόνος είναι πολλαπλάσιες φορές πιο σύντομος από το baseline καθώς γίνονται λιγότερες συγκρίσεις αλλά το recall είναι απογοητευτικό. Αντίστροφα, με μεγάλους κάδους (w=3,0x) οι κάδοι γεμίζουν με πολλά πιθανά ζευγάρια φτάνοντας το 100% recall αλλά με απογοητευτικό χρόνο σε σχέση με το baseline (1,2x). Το w=1,5x αποτελεί τη "χρυσή τομή".

4.4 Επίδραση Ακτίνας Αναζήτησης (Θ)

Τέλος, εξετάστηκε η ανθεκτικότητα του συστήματος (robustness) για διαφορετικά κατώφλια απόστασης, με τις βέλτιστες παραμέτρους (L=50, K=4, w=1.5x) στο δείγμα 100.000 x 1.000.

Πίνακας 3: Επίδραση του κατωφλίου απόστασης (θ) στην ανάκληση και τον χρόνο εκτέλεσης (L=50, K=4, w=1,5x).

θ	Recall	Speedup vs Baseline
50	100%	37,6x
150	99,1%	11,4x
180	98,8%	7,0x
200	98,9%	5,3x
250	98,6%	3,1x
300	98,3%	2,0x

Σε αυτό το μεγάλο εύρος των κατωφλίων παρατηρείται ανάκληση σταθερά άνω του 98,3%. Για “χαλαρά” θ βλέπουμε ότι ο χρόνος δεν έχει και μεγάλη βελτίωση σε σχέση με το baseline. Αυτό συμβαίνει γιατί το πλάτος του κάδου είναι δυναμικό, ενώ σε μικρά θ υπάρχει τρομερή βελτίωση. Να σημειωθεί ότι για θ = 50 υπάρχει μόνο ένα ζευγάρι που ο LSH κατάφερε να βρει 37,6 φορές πιο γρήγορα από το baseline!

4.5 Σύγκριση Κλιμακωσιμότητας (Baseline vs LSH)

Για να ελεγχθεί η κλιμακωσιμότητα της προτεινόμενης λύσης εκτελέστηκαν πειράματα με αυξήσεις στο μέγεθος της βάσης R διατηρώντας σταθερές τις παραμέτρους: θ=200, L=50, K=4, w=1,5×θ.

Πίνακας 4: Σύγκριση χρόνου εκτέλεσης και επιτάχυνσης (Speedup) μεταξύ Baseline και LSH για διαφορετικά μεγέθη του συνόλου R (θ=200, L=50, K=4, w=1,5x).

R size	S size	Baseline (s)	LSH Time (s)	Recall (%)	Speedup vs Baseline
1.000	1.000	2,31	5,59	98,9%	0,4x
10.000	1.000	27,03	10,12	98,8%	2,7x
100.000	1.000	261,42	50,3	98,9%	5,3x

Από τα αποτελέσματα του πειράματος προκύπτουν δύο κρίσιμα συμπεράσματα για το περιβάλλον των μεγάλων δεδομένων:

- Η επίδραση του αρχικού χρονικού κόστους των κατανεμημένων συστημάτων κάνει τον αλγόριθμο αισθητά πιο αργό σε μικρά σύνολα δεδομένων. Το στήσιμο του LSH, ο κατακερματισμός και η μετάδοση του ευρετηρίου στους κόμβους εργασίας απαιτεί περισσότερο χρόνο από την σάρωση και σύγκριση όλων τον δεδομένων σε μικρά σύνολα.
- Όταν ο όγκος των δεδομένων αυξάνεται ο χρόνος του baseline αυξάνεται γραμμικά και ανάλογα με αυτή την αύξηση (από 2.3s στα 27s και τέλος στα 261s, ακολουθώντας τον 10x πολλαπλασιασμό των δεδομένων) ακολουθώντας την θεωρητική τετραγωνική πολυπλοκότητα O(|R| x |S|). Στον αντίποδα ο χρόνος του LSH αυξάνεται με πιο ήπιο υπογραμμικό τρόπο. Ως αποτέλεσμα ο LSH κρατάει σταθερά υψηλό recall και γίνεται όλο και ταχύτερος σε σχέση με τον baseline όσο αυξάνονται τα δεδομένα (5,2x επιτάχυνση σε σχέση με τον baseline σε 100.000 x 1.000 συγκρίσεις).

4.6 Εκτέλεση σε Πλήρη Κλίμακα

Στο τελικό στάδιο της πειραματικής αποτίμησης της λύσης εφαρμόστηκε ο αλγόριθμος στο καθολικό σύνολο δεδομένων SIFT1M. Στο SIFT1M περιέχονται τα σύνολα R και S με 1.000.000 και 10.000 128-διάστατα διανύσματα. Οι αναζητήσεις που καλούνται να γίνουν αγγίζουν τα 10 δισεκατομμύρια πιθανά ζευγάρια. Βάσει των μετρήσεων της ενότητας 4.5 η baseline μέθοδος χρειάστηκε 261,4 δευτερόλεπτα για 100 εκατομμύρια συγκρίσεις, συνεπώς ακολουθώντας τη γραμμική πορεία της baseline μεθόδου η σύγκριση για το πλήρες SIFT1M θα απαιτούσε περίπου 26.140 δευτερόλεπτα (7,26 ώρες ασταμάτητου υπολογισμού).

Η εκτέλεση του LSH με κατανεμημένο τρόπο σε δύο πυρήνες και με μόλις 10 GB διαθέσιμης μνήμης αξιοποιώντας την αρχιτεκτονική Map-Side Broadcast Join, χρειάστηκε μόλις 4.278

δευτερόλεπτα (~71,30 λεπτά) βρίσκοντας 1.767.676 ζευγάρια των οποίων η Ευκλείδεια απόσταση είναι μικρότερη ή ίση του 200.

Συνεπώς ακόμα και σε ένα "αδύναμο" κατανεμημένο σύστημα υπάρχει βελτίωση του χρόνου 6,1 φορές έναντι του εξαντλητικού baseline μοντέλου. Ο LSH επιλύει τα προβλήματα σε Big Data περιβάλλον σε απολύτως βιώσιμους χρόνους μη πέφτοντας "θύμα" της κατάρας της διαστατικότητας.

5. Συμπεράσματα

Ή εργασία αυτή μελέτησε μια λύση για το πρόβλημα της εύρεσης όμοιων διανυσμάτων σε περιβάλλον Μεγάλων Δεδομένων μέσω μιας κατανεμημένης και προσεγγιστικής προσέγγισης. Η παραδοσιακή μέθοδος ελέγχου όλων τον διαθέσιμων ζευγαριών υπόκειται σε τετραγωνική πολυπλοκότητα O(|R| x |S|) κάτι που την καθιστά αδύνατη σε μεγάλα σύνολα. Επιπλέων διακατέχεται από την κατάρα της διαστατικότητας. Αυτά τα προβλήματα λύνονται επαρκώς πετυχαίνοντας ακρίβειες μεγαλύτερες από 98% με προσεγγιστικό αλγόριθμο βασισμένο στο p-stable Locality-Sensitive Hashing (LSH)[2].

Ενσωματώνοντας στρατηγικές, όπως το Map-Side Broadcast Join στο πλαίσιο του Apache Spark, την τμηματική επεξεργασία (chunking) και τις πράξεις διανυσμάτων μέσω NumPy ο LSH λειτούργησε άψογα. Η πειραματική διαδικασία ανέδειξε την απόδοση του αλγορίθμου για διαφορετικά κατώφλια και μεγέθη συνόλων διατηρώντας την ανάκληση σε επίπεδα σχεδόν 99%. Ταυτόχρονα διαπιστώθηκε η υπογραμμική συμπεριφορά του αλγορίθμου, ο οποίος χρειάστηκε μόλις 71,3 λεπτά έναντι των ~7,26 ωρών της μεθόδου baseline επιταχύνοντας την διαδικασία 6,1 φορές ακόμα και στο "αδύναμο" περιβάλλον του Google Colab με 2 πυρήνες και 10GB μνήμης ram που χρησιμοποιήσαμε.

Συμπερασματικά η θεωρητική μαθηματική προσέγγιση του LSH αποδείχθηκε στην πράξη καθιστώντας τον ικανό να λύσει προβλήματα διανυσματικής σύζευξης τεράστιας κλίμακας σε απολύτως βιώσιμους χρόνους.

6. Μελλοντικές Επεκτάσεις

Μελλοντικά το σύστημα που δημιουργήθηκε θα μπορούσε να ενταχθεί σε παραγωγικά περιβάλλοντα. Για την επίτευξη αυτού το σύστημα πρέπει να αυτοματοποιηθεί πλήρως μέσω δυναμικής παραμετροποίησης αλλά και υποστήριξης δεδομένων πραγματικού χρόνου.

- **Δυναμική Παραμετροποίηση:** Η ενσωμάτωση τεχνολογιών μηχανικής μάθησης με σκοπό την αυτόματη και δυναμική επιλογή των βέλτιστων παραμέτρων θα καταστήσει το σύστημα αποδοτικό ακόμα και σε δεδομένα με άλλες κατανομές και διαστάσεις που μπορεί να του δοθούν. Αυτό μπορεί να επιτευχθεί με στοχευμένη δειγματοληψία για την κατανόηση των στατιστικών μεγεθών του συνόλου αλλά και με εφαρμογή μαθηματικών μοντέλων κόστους.
- **Υποστήριξη Δεδομένων Πραγματικού Χρόνου:** Με την αξιοποίηση τεχνολογιών όπως Apache Spark Structured Streaming σε συνδυασμό με το Apache Kafka η παρούσα αρχιτεκτονική θα μπορέσει να διαχειριστεί συνεχείς ροές δεδομένων. Με αυτόν τον τρόπο ανακαλύπτονται άμεσα τα όμοια διανύσματα που εισέρχονται στο σύστημα. Καθιστόντας την αρχιτεκτονική αυτή να μπορεί να χρησιμοποιηθεί σε εφαρμογές αναλύσεων σε πραγματικό χρόνο.

7. Δήλωση Χρήσης Εργαλείων Τεχνητής Νοημοσύνης

Κατά τη διάρκεια εκπόνησης της παρούσας εργασίας, αξιοποιήθηκαν εργαλεία Τεχνητής Νοημοσύνης (Μεγάλα Γλωσσικά Μοντέλα) αυστηρά σε επικουρικό ρόλο, με σκοπό την υποστήριξη της ερευνητικής και προγραμματιστικής διαδικασίας. Η χρήση τους επικεντρώθηκε στην αρχική αναζήτηση βιβλιογραφικής κατεύθυνσης για την αρχιτεκτονική της λύσης, καθώς και στον εντοπισμό και τη διόρθωση σφαλμάτων (debugging) κατά τη συγγραφή του κώδικα. Επιπλέον, το AI χρησιμοποιήθηκε ως βοηθητικό εργαλείο για την ταχύτερη προσαρμογή και παραμετροποίηση των διαφόρων πειραματικών σεναρίων, έχοντας πάντα ως θεμέλιο τον βασικό κώδικα και τη λογική που είχε ήδη στηθεί από τον συγγραφέα. Ο τελικός σχεδιασμός, η επιλογή των παραμέτρων, η κριτική ανάλυση των αποτελεσμάτων και η διαμόρφωση της αναφοράς αποτελούν αποκλειστικό πνευματικό έργο του συγγραφέα. **Διευκρινίζεται, τέλος, ότι η παρούσα παράγραφος δήλωσης αποτελεί το μοναδικό σημείο ολόκληρου του εγγράφου όπου χρησιμοποιήθηκε Τεχνητή Νοημοσύνη για την απευθείας παραγωγή και σύνταξη κειμένου.**

8. Βιβλιογραφία

[1] P. Indyk and R. Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 604–613, 1998.

[2] M. Datar, N. Immorlica, P. Indyk, and V. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the 20th Annual Symposium on Computational Geometry (SoCG)*, pp. 253–262, 2004.

[3] R. Vernica, M. J. Carey, and C. Li. Efficient parallel set-similarity joins using MapReduce. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pp. 495–506, 2010.

[4] J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

[5] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012.